School of Science and Engineering

MIDTERM EXAMINATION

Term 2, 2025

CSC1001 Introduction to Computer Science

Examination Duration: 120 minutes

Reading Time: 10 minutes

This examination has __5__ questions.

The score distribution of the 5 question is Question 1(40%), Question 2(20%), Question 3(10%), Question 4(15%), Question 5(15%)

Exam Conditions:

This is a FORMAL Examination

This is a RESTRICTED OPEN BOOK Exam. Maximum of one (1) sheet of handwritten notes double-sided are permitted

Materials Permitted In The Exam Venue:

Maximum of one (1) sheet of handwritten notes double sided are permitted. **NO OTHER MATERIALS PERMITTED**

Calculators without the functionalities of programming and file storage are permitted.

Materials To Be Supplied To Students:

1 × 6 Page Answer Booklet

You must write all your answers in your answer book.

Question 1. (40%)

Please describe what will be printed for each program as below.

	Program	Printed Results

1) 3%	`x = (2**3 % 5) // 2 + float("77".replace("7", "1"))`
	`print(x)`
	
	Explanation: 2**3 % 5 is 3, and 3 // 2 is 1. Replacing each 7 in "77" with 1 gives "11", or 11.0; the sum is 12.0.
		Answer: 12.0

2) 3%	`s1, s2, s3 = eval("15-3"), str(15-3), "15-3"`
	`print(s1 + len(s2) + len(s3))`
	
	Explanation: eval("15-3") gives 12; str(15-3) is "12"; s3 is "15-3". Their string lengths are 2 and 4, so 12 + 2 + 4 = 18.
		Answer: 18

3) 3%	`t = [10, 20, "python", False, 80]`
	`print(t[2][2:5].upper(), t[0:2])`
	
	Explanation: t[2] is "python" and [2:5] selects "tho", whose uppercase form is "THO". t[0:2] selects [10, 20].
		Answer: THO [10, 20]

4) 3%	`x = 10`
	`y = []`
	
	`if (not x > 5) or (y and x % 2 == 0):`
	`  print("csc1001")`
	`elif len(y) == 0 and x > 8:`
	`  print("hello")`
	`else:`
	`  print("world")`
	
	Explanation: x is 10 and y is empty. The first condition is false; len(y)==0 and x>8 are both true, so the elif branch prints hello.
		Answer: hello

5) 3%	`n = 12`
	`i = 1`
	`while i < n:`
	`  print(i, end="*")`
	`  i = i * 2`
	
	Explanation: i starts at 1 and doubles after each print. The values 1, 2, 4, and 8 are below 12; 16 ends the loop.
		Answer: 1*2*4*8*

<table>
<tr>
<td>6) 4%</td>
<td>

```
def log_event(event, cache=[], max_size=1):
    cache.append(event)
    if len(cache) > max_size:
        cache[:] = cache[1:]
    print(cache)

log_event("start")
log_event("finish", max_size=2)
```

Explanation: The default cache list is created once and shared by calls that omit the cache argument. The first call appends start and prints ['start']. The second call appends finish to that same list. Its length is 2, which is not greater than max_size=2, so nothing is removed; it prints ['start', 'finish'].

Tip: Mutable default arguments, such as cache=[], are created once when the function is defined and reused by calls that omit the argument. To avoid sharing a list accidentally, use None as the default and create a new list inside the function.

Safer version (creates a new list when cache is omitted):

```
def log_event(event, cache=None, max_size=1):
    if cache is None:
        cache = []

    cache.append(event)
    if len(cache) > max_size:
        cache[:] = cache[1:]
    print(cache)
```

</td>
<td>Answer: ['start']
['start', 'finish']</td>
</tr>
<tr>
<td>7) 4%</td>
<td>

```
data = "student_ID_12345.csv"
course = ""
recording = False
for char in data:
    if char == '_':
        recording = not recording
        continue
    if char == '.':
        break
    if recording:
        course += char
        print(course)
```

Explanation: The first underscore toggles recording on and is skipped. The letters I and D are then added one at a time, so the program prints I and then ID. The second underscore toggles recording off; the digits are ignored, and the period ends the loop.

</td>
<td>Answer: I
ID</td>
</tr>
</table>

8) 3%	```for i in range(3):
	    if i == 2:
	        print(i, end=",")
	        break
	    else:
	        print("Loop finished")```
	
	Explanation: For i=0 and i=1, the condition i == 2 is false, so this else branch prints "Loop finished" on separate lines. At i=2, print(i, end=",") prints 2 followed by a comma without a newline, and break exits the loop. There is no for-else clause here.
		Answer:
		Loop finished
		Loop finished
		2,

9) 3%	```s = "user-tongxin-li-admin@example.com" print(s.split("@")[0].split("-")[-1])``` Explanation: Splitting at @ selects the username. Splitting on hyphens and taking the final piece returns admin. print() displays this string without quotation marks.	Answer: admin
10) 4%	```counts = {'a': 5, 'b': 10} counts['c'] = counts.get('c', 0) + 1 print(counts['a'], counts['c']) print('b' in counts)``` Explanation: The missing count for c defaults to 0 and is incremented to 1. The first print() displays 5 and 1 separated by a space, then a newline. The second print() displays True because b is a key in counts.	Answer: 5 1 True
11) 3%	```h = open('example.txt', 'w') h.write('Hello') h.close() h = open('example.txt', 'r') s = h.read() print('First read:', s) s = h.read() print('Second read:', s)``` Explanation: The first read() consumes Hello and leaves the file position at the end. The next read() returns the empty string. The two print() calls produce separate lines without quotation marks. The second line ends with one space after the colon, because print() inserts its default separator before the empty string.	Answer: First read: Hello Second read:

12) 4%	```
	d = {'b':1, 'a':2, 'c':0}
	lst = []
	for k, v in d.items():
	    if v > 0:
	        lst.append(k)
	        lst.sort()
	        print(lst)
	``
	Explanation: lst.sort() and print(lst) are inside the if block, which is inside the loop. Dictionary items are visited in insertion order: b, a, then c. For b, the list becomes ['b'] and is printed. For a, appending and sorting produces ['a', 'b'], which is printed on the next line. The value for c is 0, so that iteration neither changes nor prints the list.
		Answer:
		['b']
		['a', 'b']

Question 2. (5*4% = 20%)

Please answer the following short questions.

1) Provide the Binary and Hexadecimal representations of the decimal number 29.25.

Answer: Binary = 11101.01; hexadecimal = 1D.4.

Explanation: 29 is 11101_2 and 0.25 is $.01_2$. Grouping into four-bit digits gives 0001 1101 . 0100 = $1D.4_{16}$.

2) Please list two advantages of using binary numbers in modern computer systems.

Answer: Binary states are reliable in electronic circuits and support simple, efficient Boolean logic.

Explanation: Two voltage states are easy to distinguish despite electrical noise, and Boolean operations map directly to digital circuits.

3) Please describe the differences between arguments and parameters.

Answer: Parameters are names in a function definition; arguments are values passed in a function call.

Explanation: In def f(x), x is a parameter; in f(3), 3 is an argument.

4) Will the following be implemented without causing an error? (Yes/No)

```
data = ("apple", 5, "orange")
print(data[0])
data[1] = 10
print(data)
```

Answer: No. It first prints apple, then raises a TypeError at data[1] = 10. The final print(data) is not executed.

Explanation: data is a tuple. Reading data[0] is valid and print() displays apple without quotation marks. Tuples do not support item assignment, so data[1] = 10 raises TypeError and stops execution before print(data).

Explain the difference between a list and a tuple in Python. Provide one reason to use a tuple instead of a list.

Answer: A list is mutable; a tuple is immutable. A tuple is useful for fixed data that should not change.

Explanation: Lists allow item assignment, addition, and removal. A tuple does not allow its stored references to be replaced, added, or removed. If a tuple contains a mutable object such as a list, that object can still be modified; tuple immutability does not make every contained object immutable.

5) What is the purpose of the return keyword in a function? What happens if a function does not have a return statement?

Answer: return sends a result to the caller and ends the function. Without an explicit return, the function returns None.

Explanation: The caller can use the returned value. Reaching the end without returning a value produces None.

Question 3. (10%)

Write a Python program that plays a number-guessing game with a user. The game should follow these rules:

- The computer fixes a **secret random integer** between 1 and 50.
- The user gets a maximum of **5** attempts to guess the number.
- After each guess, the program must provide feedback to the user, telling them if their guess was "**Too high**!" or "**Too low!**".

- If the user guesses correctly, the game ends, and the program prints a success message, like "**Correct! You guessed it in [N] attempts.**" where N is the total number of attempts.
- If the user runs out of attempts, the game ends, and the program should reveal the secret number, printing "**Sorry, you've run out of attempts. The secret number was [X].**" where X is the correct integer.

Answer and explanation:

```
import random
secret = random.randint(1, 50)
for attempt in range(1, 6):
  guess = int(input("Guess a number from 1 to 50: "))
  if guess == secret:
    print(f"Correct! You guessed it in {attempt} attempts.")
    break
  elif guess > secret:
    print("Too high!")
  else:
    print("Too low!")
else:
  print(f"Sorry, you've run out of attempts. The secret number was {secret}.")
```

randint(1, 50) includes both endpoints. The loop allows at most five guesses. It gives high/low feedback, breaks on success, and runs its else clause only if all tries finish without a break.

Question 4. (15%)

Please first explain the usage of the built-in string method **find()**. Then, write a function **myFind**(*main_string*, *sub_string*):

- *main_string* is the string to be searched.
- *sub_string* is the string to search for.

- The function should return the **ending index** of the first occurrence of *sub_string* within *main_string*.
- If *sub_string* is not found, the function should return -1.

You may not use the built-in find() or index() methods in your implementation.

(e.g., if *main_string* is "hello world csc1001" and *sub_string* is "world", the result should be 10)

Answer and explanation: str.find(sub_string[, start[, end]]) returns the zero-based starting index of the first occurrence, or -1 if absent. Optional start and end restrict the search range, with end excluded. The required myFind() instead returns the inclusive ending index: world occupies indices 6 through 10 in the example, so it returns 10. This solution returns 0 for an empty sub_string, following the explicit rule in the 2025 Fall paper; the question printed here does not specify this case.

```
def myFind(main_string, sub_string):
    if sub_string == "":
        return 0

    n = len(main_string)
    m = len(sub_string)

    for i in range(n - m + 1):
        if main_string[i:i + m] == sub_string:
            return i + m - 1

    return -1
```

n and m are the lengths of the main string and substring. An empty substring returns 0 immediately. The loop checks possible starting positions from 0 through n - m. The slice main_string[i:i + m] contains m characters because its right endpoint is excluded. If it matches, i + m - 1 is the index of the final matched character. Returning immediately ensures the first occurrence is used. If no match exists, including when the substring is longer than the main string, the function returns -1.

Question 5. (15%)

You are in charge of counting the votes for a class election. You have a file, **votes.txt**, where every line contains the name of the candidate who received a vote.

Example **votes.txt** content:
Alice
Bob
Alice
Charlie
Alice
Bob
…

Please write a Python program that:

- Reads the **votes.txt** file.
- Counts the total number of votes for each candidate.
- Creates a new file named **results.txt** that lists each candidate and their total vote count, sorted alphabetically by the candidate's name.

Answer and explanation:

```
votes = {}
with open("votes.txt", "r") as source:
  for line in source:
    name = line.strip()
    if name:
      votes[name] = votes.get(name, 0) + 1
with open("results.txt", "w") as output:
  for name in sorted(votes):
    output.write(f"{name} {votes[name]}\n")
```

strip() removes leading and trailing whitespace, including the line ending. Empty or whitespace-only lines are skipped. The dictionary tracks vote totals, and sorted(votes) visits candidate names in alphabetical order for the names shown. Each candidate and count is written on its own line. The sample results are Alice 3, Bob 2, and Charlie 1.

END OF EXAMINATION